



INSTITUTO SUPERIOR TECNOLÓGICO
RUMIÑAHUI
CON CONDICIÓN DE SUPERIOR UNIVERSITARIO



Sistemas y Gestión de Data

Reutilización del software

S7-Tarea 2P (Avance)

**Desarrollo de un Tablero Vehicular Inteligente con
Monitoreo en Tiempo Real y Almacenamiento de Datos:
Implementación Full-Stack con React, Node.js y MongoDB**

**GRANJA ALENCASTRO PABLO
NAVARRO PÉREZ MARCELO**

Cuarto Semestre

Link al proyecto en One Drive institucional: https://ister-my.sharepoint.com/:f/g/personal/pablo_granja_ister_edu_ec/IgCiMqAhaGcjQa1tW3M6VoGOAXJ6eezogPL9-O3oF1oVKOM?e=KgL8Tl

[Link](#)

Tabla de contenido

Actividad:	1
Título del Trabajo:	1
Descripción General:	1
Objetivos:	1
Objetivo del Trabajo:	1
Objetivos específicos	1
Desarrollo:	2
Resumen de Pasos de Desarrollo	2
Productos Generados	3
Estructura del Programa (Arquitectura de Carpetas)	6
Componentes del Backend	6
Componentes del Frontend	6
Anexos:	7
Backend:	7
VehicleData.js	7
data.js	8
server.js	8
Frontend:	9
components (amperaje, gasolina, lucesadvertencia, odómetro, tacómetro, temperatura, velocímetro y voltaje)	9
useVehicleData.js	12
App.jsx	13

Actividad:

Título del Trabajo:

Desarrollo de un Tablero Vehicular Inteligente con Monitoreo en Tiempo Real y Almacenamiento de Datos: Implementación Full-Stack con React, Node.js y MongoDB.

Descripción General:

El presente proyecto comprende el diseño e implementación de un tablero vehicular digital (dashboard) que permita visualizar los indicadores principales de un vehículo, tales como velocidad, revoluciones del motor (RPM), temperatura, voltaje, amperaje, nivel de combustible y kilometraje, además, el sistema incorpora un panel de testigos luminosos (luces de advertencia) y un módulo de almacenamiento de datos en una base de datos, con el fin de registrar el historial de mediciones para su posterior análisis.

Objetivos:

Objetivo del Trabajo:

- Desarrollar un tablero vehicular digital basado en una arquitectura web full-stack que permita la visualización en tiempo real de los indicadores de instrumentación y el almacenamiento histórico de datos, utilizando tecnologías modernas como React, Node.js y MongoDB.

Objetivos específicos

- Diseñar e implementar la interfaz gráfica del tablero vehicular mediante componentes reutilizables en React y gráficos vectoriales (SVG), para representar los indicadores físicos del vehículo (velocidad, RPM, temperatura, voltaje, gasolina y odómetro), incluyendo un panel de testigos luminosos de advertencia.

- Construir una arquitectura de servidor y base de datos utilizando Node.js, Express y MongoDB, cómo medio para estructurar la información de las mediciones vehiculares a través de una API, garantizando la captura automática y el almacenamiento de los registros.
- Integrar el frontend con el backend mediante un sistema de simulación de datos dinámicos, con el fin de validar el flujo de información en tiempo real, demostrar la capacidad de respuesta del sistema ante variaciones continuas de los sensores y preparar las bases para la futura integración con hardware físico (Arduino).

Desarrollo:

Resumen de Pasos de Desarrollo

El proceso de construcción del tablero vehicular se ejecutó de manera incremental, siguiendo una metodología ágil dividida en las siguientes etapas:

1. Configuración del Entorno Frontend: Inicialización del proyecto con Vite y React, e integración de Tailwind CSS para el diseño de estilos responsivos y modernos.
2. Diseño de Componentes Visuales (UI): Desarrollo de medidores circulares personalizados utilizando SVG puro (sin librerías externas) para garantizar un alto rendimiento. Se crearon los componentes de Velocímetro, Tacómetro, Temperatura, Voltaje, Amperaje y Gasolina, junto con el Odómetro digital y el panel de testigos luminosos.
3. Implementación de Lógica de Negocio y Simulación: Creación de hooks personalizados en React (useVehicleData) para generar y gestionar la simulación de datos en tiempo real, aplicando fórmulas matemáticas para emular el comportamiento físico de un vehículo (ej. consumo de combustible, variación de RPM).
4. Desarrollo del Backend: Configuración de un servidor independiente con Node.js y Express. Se diseñó el esquema de datos utilizando Mongoose para modelar la información vehicular en MongoDB.
5. Integración Full-Stack y Persistencia: Conexión del frontend con el backend mediante peticiones HTTP (fetch API). Se implementó un sistema de guardado periódico para enviar los datos simulados a la base de datos, validando el flujo completo de información.

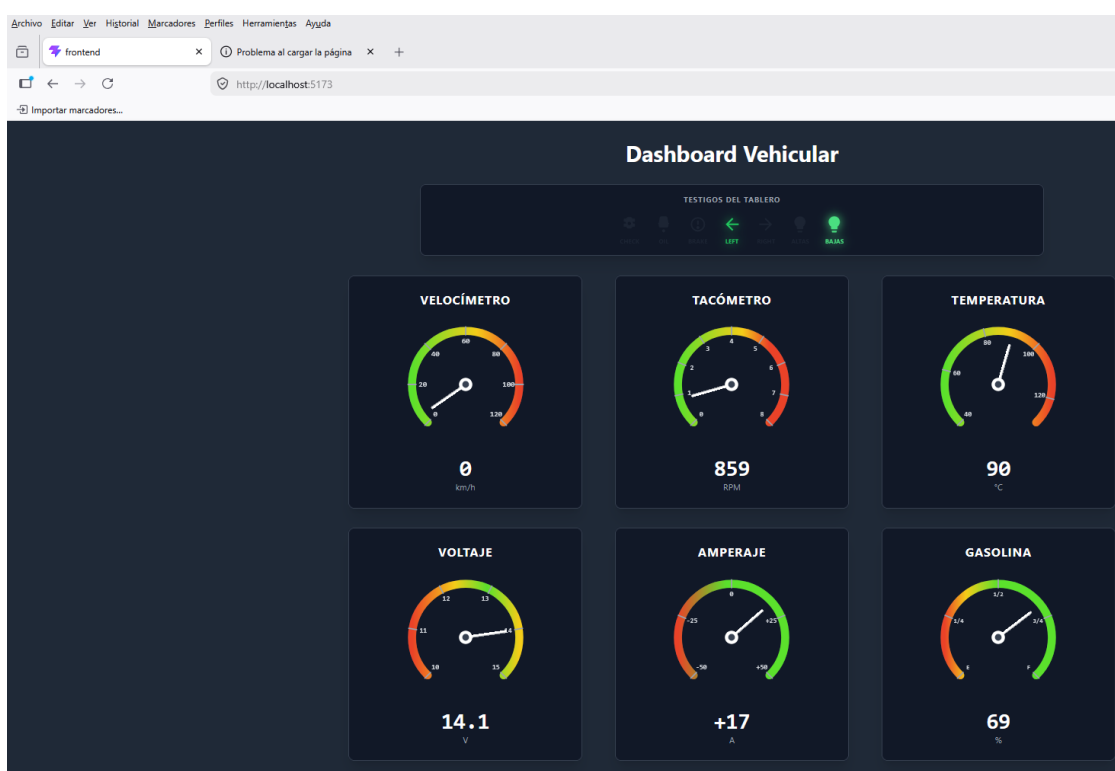
Productos Generados

Como resultado de esta fase de desarrollo, se han obtenido los siguientes productos tangibles y funcionales:

1. Interfaz Gráfica de Usuario (GUI) del Tablero: Una aplicación web responsiva que visualiza 7 indicadores vehiculares y un panel de 7 testigos de advertencia con retroalimentación visual en tiempo real.

Figura 1

Interfaz gráfica del panel de instrumentos generado para el almacenamiento de datos de un vehículo



2. Motor de Simulación de Datos: Un módulo de software capaz de generar flujos continuos de datos coherentes y realistas que alimentan la interfaz.

Figura 2

Código utilizado para definir las características visuales de los indicadores o controles.

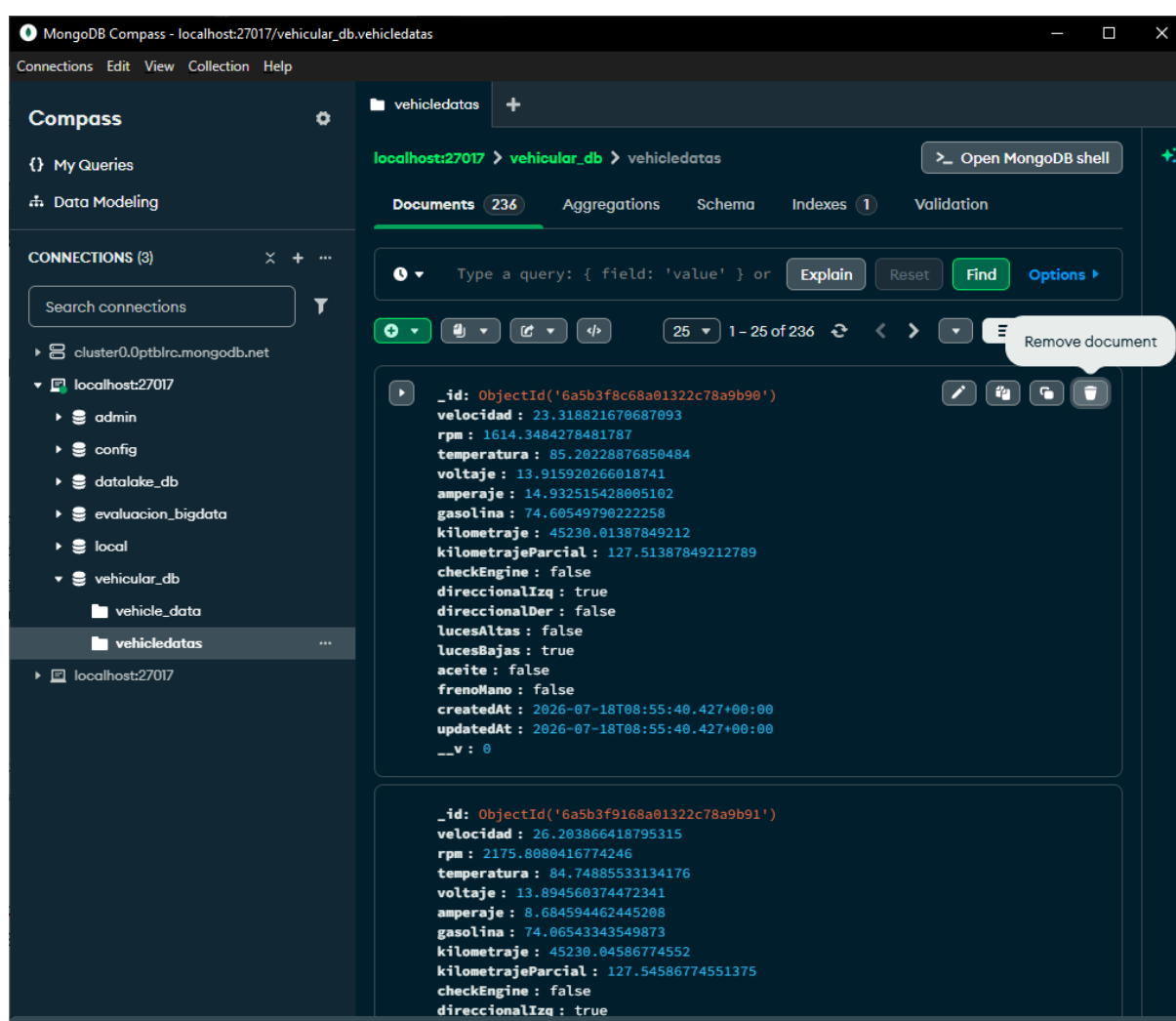
```

1  import { useState, useEffect, useRef } from 'react';
2
3  export function useVehicleData() {
4    const [data, setData] = useState({
5      velocidad: 0,
6      rpm: 800,
7      temperatura: 85,
8      voltaje: 13.8,
9      amperaje: 15.5,
10     gasolina: 75,
11     kilometraje: 45230,
12     kilometrajeParcial: 127.5,
13     checkEngine: false,
14     direccionalIzq: false,
15     direccionalDer: false,
16     lucesAltas: false,
17     lucesBajas: true,
18     aceite: false,
19     frenoMano: false
20   });
21
22   // Usamos un ref para tener siempre el valor más reciente de los datos
23   const dataRef = useRef(data);
24
25   // Actualizamos el ref cada vez que los datos cambian
26   useEffect(() => {
27     dataRef.current = data;
28   }, [data]);
29
30   // 1. Simulación visual (se actualiza cada 500ms para que las agujas se muevan)
31   const intervaloSimulacion = setInterval(() => {
32     setData(prev => {
33       const velocidadCambio = (Math.random() - 0.5) * 10;
34       let nuevaVelocidad = Math.max(0, Math.min(120, prev.velocidad + velocidadCambio));
35
36       const rpmBase = 800 + (nuevaVelocidad / 120) * 5200;
37       let nuevaRpm = Math.max(800, Math.min(8000, rpmBase + (Math.random() - 0.5) * 1000));
38
39       let nuevaTemp = Math.max(80, Math.min(100, prev.temperatura + (Math.random() - 0.5) * 1));
40       let nuevoVolt = Math.max(13.5, Math.min(14.5, prev.voltaje + (Math.random() - 0.5) * 0.1));
41       let nuevoAmp = Math.max(-10, Math.min(30, prev.amperaje + (Math.random() - 0.5) * 0.5));
42
43       let nuevaGas = prev.gasolina - (Math.random() * 0.1);
44       if (nuevaGas < 0) nuevaGas = 100;
45
46       const kmAumento = (nuevaVelocidad / 3600) * 0.5;
47       const nuevoKmTotal = prev.kilometraje + kmAumento;
48       const nuevoKmParcial = prev.kilometrajeParcial + kmAumento;
49
50       return {
51         ...prev,
52         velocidad: nuevaVelocidad,
53         rpm: nuevaRpm,
54         temperatura: nuevaTemp,
55         voltaje: nuevoVolt,
56         amperaje: nuevoAmp,
57         gasolina: nuevaGas,
58         kilometraje: nuevoKmTotal,
59         kilometrajeParcial: nuevoKmParcial,
60         direccionalIzq: !prev.direccionalIzq // Parpadeo
61       };
62     });
63   }, 500);
64
65   // 2. Guardado en Base de Datos (se envía cada 5 segundos para no saturar la BD)
66   const intervaloGuardado = setInterval(() => {
67     fetch('http://localhost:3001/api/data/save', {
68       method: 'POST',
69       headers: { 'Content-Type': 'application/json' },
70       body: JSON.stringify(dataRef.current)
71     })
72     .then(response => response.json())
73     .then(resultado => console.log('Guardado en BD:', resultado.mensaje))
74     .catch(error => console.error('Error al guardar:', error));
75   }, 5000); // Cada 5000 milisegundos (5 segundos)
76
77   // Limpiar intervalos al desmontar el componente
78   return () => {
79     clearInterval(intervaloSimulacion);
80     clearInterval(intervaloGuardado);
81   };
82 }, []);
83
84 return data;
85
86 }
  
```

3. API REST de Monitoreo Vehicular: Un servicio backend con endpoints documentados (POST /save, GET /latest) para la recepción y consulta de telemetría.
4. Base de Datos Histórica: Una base de datos NoSQL (MongoDB) poblada con registros de telemetría vehicular, incluyendo marcas de tiempo automáticas para futuros análisis de tendencias.

Figura 3

Captura de pantalla de la base de datos NoSQL en Mongo DB, donde se observa el almacenamiento de los valores generados en la aplicación cada 5 segundos.



5. Código Fuente Modular: Un repositorio de código estructurado bajo principios de reutilización de software, con componentes desacoplados y fácilmente escalables.

Estructura del Programa (Arquitectura de Carpetas)

El proyecto está organizado bajo una arquitectura de monorepo, separando claramente las responsabilidades del cliente (Frontend) y del servidor (Backend). La estructura de directorios es la siguiente:

Figura 4

Captura de pantalla de la estructura de las carpetas en el explorador de Windows.

Nombre	Fecha de modificación	Tipo	Tamaño
backend	2026-07-18 03:29	Carpeta de archivos	
frontend	2026-07-18 03:19	Carpeta de archivos	
localhost_files	2026-07-17 23:27	Carpeta de archivos	
node_modules	2026-07-17 23:34	Carpeta de archivos	
localhost	2026-07-17 23:27	Brave HTML Docu...	9 KB
package	2026-07-17 23:34	Archivo de origen ...	1 KB
package-lock	2026-07-17 23:34	Archivo de origen ...	40 KB

Componentes del Backend

Figura 5

Captura de pantalla de la estructura de las carpetas y archivos generados en el backend.

File Edit Selection View ...

EXPLORER

2DO PARCIAL AVANCE PANEL DE INSTR...

backend

models

VehicleData.js

node_modules

routes

data.js

.env

npm

package-lock.json

package.json

server.js

Capa de Lógica y Datos - Node.js

Models: Esquemas de la base de datos

VehicleData.js: Modelo Mongoose

Routes: Endpoints de la API REST

data.js: Rutas para guardar y consultar

.env: Variables de entorno (Puerto, URI de Mongo)

server.js: Servidor Express principal

package.json: Dependencias del backend

Componentes del Frontend

Figura 6

Captura de pantalla de la estructura de las carpetas y archivos generados en el frontend.



Anexos:

Backend:

VehicleData.js

Es el Esquema de la base de datos, ordenándole a MongoDB exactamente qué tipo de datos se va a guardar, qué campos son obligatorios y cómo deben estructurarse.

```

JS VehicleData.js X
backend > models > JS VehicleData.js > ...
1  const mongoose = require('mongoose');
2
3  const vehicleDataSchema = new mongoose.Schema({
4    velocidad: { type: Number, required: true },
5    rpm: { type: Number, required: true },
6    temperatura: { type: Number, required: true },
7    voltaje: { type: Number, required: true },
8    amperaje: { type: Number, required: true },
9    gasolina: { type: Number, required: true },
10   kilometraje: { type: Number, required: true },
11   kilometrajeParcial: { type: Number, required: true },
12   checkEngine: { type: Boolean, default: false },
13   direccionalIzq: { type: Boolean, default: false },
14   direccionalDer: { type: Boolean, default: false },
15   lucesAltas: { type: Boolean, default: false },
16   lucesBajas: { type: Boolean, default: false },
17   aceite: { type: Boolean, default: false },
18   frenoMano: { type: Boolean, default: false }
19 }, {
20   timestamps: true // Esto guarda automáticamente la fecha y hora (c
21 });
22
23 module.exports = mongoose.model('VehicleData', vehicleDataSchema);

```

data.js

Actúa como el controlador de tráfico (las rutas de la API), es el intermediario que recibe las peticiones del frontend, decide qué hacer con esos datos y se comunica con la base de datos (MongoDB).

```
JS data.js X
backend > routes > JS data.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const VehicleData = require('../models/VehicleData');
4
5  // Ruta 1: Guardar datos del vehículo
6  router.post('/save', async (req, res) => {
7    try {
8      // Creamos un nuevo registro con los datos que nos envía el frontend
9      const nuevoRegistro = new VehicleData(req.body);
10     await nuevoRegistro.save();
11     res.status(201).json({ mensaje: 'Datos guardados correctamente' });
12   } catch (error) {
13     res.status(500).json({ error: error.message });
14   }
15 });
16
17 // Ruta 2: Obtener el último registro guardado
18 router.get('/latest', async (req, res) => {
19   try {
20     // Buscamos el registro más reciente ordenado por fecha
21     const ultimoRegistro = await VehicleData.findOne().sort({ createdAt: -1 });
22     res.json(ultimoRegistro);
23   } catch (error) {
24     res.status(500).json({ error: error.message });
25   }
26 });
27
28 module.exports = router;
```

server.js

Es el punto de entrada (Entry Point) o el cerebro del servidor, su función es inicializar la aplicación, configurar las reglas de comunicación (middleware), conectar la base de datos y poner a la escucha el servidor en un puerto específico.

```

JS serverjs X
backend > JS serverjs > ...
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const cors = require('cors');
4  require('dotenv').config();
5
6  // 1. Importamos las rutas que acabamos de crear
7  const dataRoutes = require('./routes/data');
8
9  const app = express();
10
11 // Middleware
12 app.use(cors());
13 app.use(express.json());
14
15 // 2. Conectamos las rutas a la dirección /api/data
16 app.use('/api/data', dataRoutes);
17
18 // Ruta de prueba (para saber que el servidor funciona)
19 app.get('/api/test', (req, res) => {
20   res.json({ mensaje: 'El backend está funcionando!' });
21 });
22
23 // Conexión a MongoDB
24 mongoose.connect(process.env.MONGODB_URI)
25   .then(() => {
26     console.log('Conectado a MongoDB correctamente');
27   })
28   .catch((error) => {
29     console.error('Error conectando a MongoDB:', error);
30   });
31
32 // Iniciar servidor
33 const PORT = process.env.PORT || 3001;
34 app.listen(PORT, () => {
35   console.log(`Servidor corriendo en http://localhost:${PORT}`);
36 });

```

Frontend:

components (amperaje, gasolina, lucesadvertencia, odómetro, tacómetro, temperatura, velocímetro y voltaje)

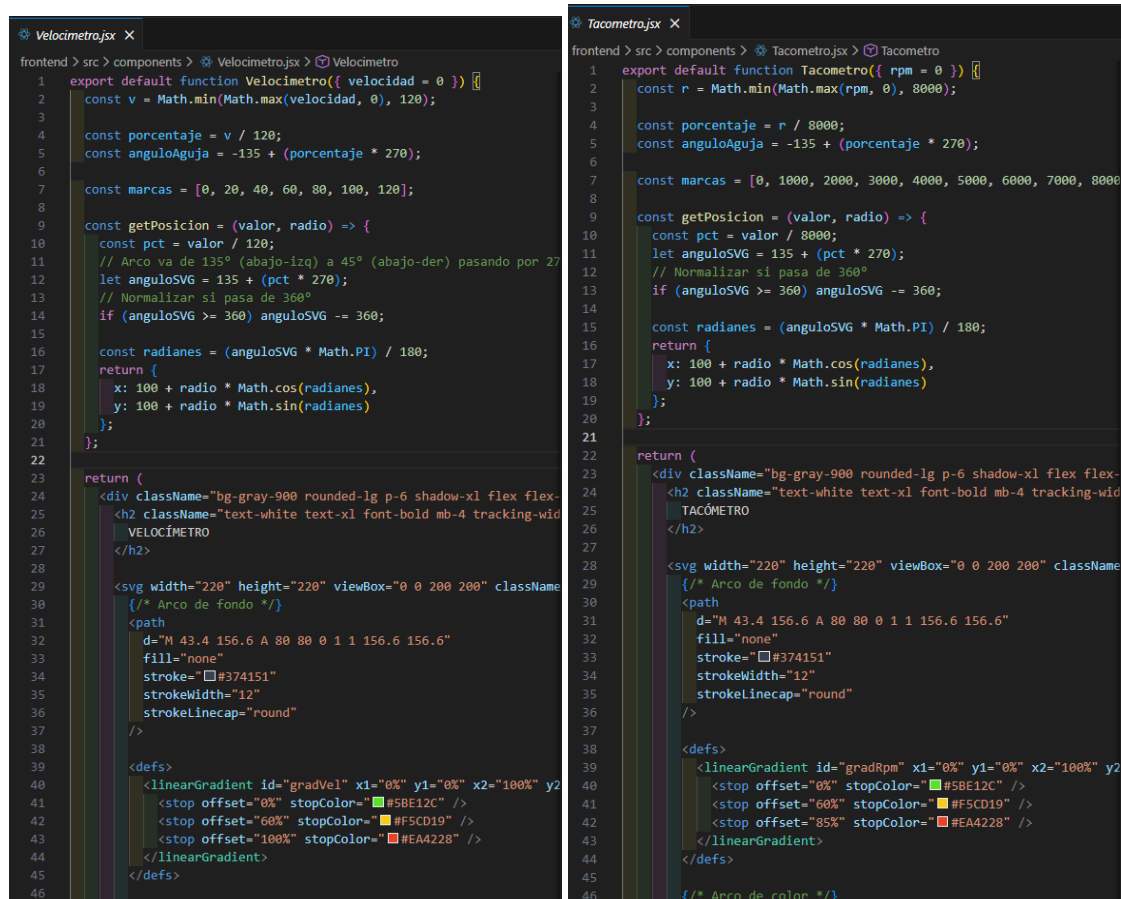
1. **Medidores Circulares con Aguja:** Estos componentes comparten la misma arquitectura: un arco SVG degradado, marcas de referencia, una aguja animada y un display digital, todos reciben un valor numérico como prop y lo normalizan a un ángulo de rotación.

Componente	Archivo	Rango de Medición	Prop que Recibe	Función Específica
Velocímetro	Velocimetro.jsx	0 - 120 km/h	velocidad	Muestra la velocidad actual del vehículo con marcas cada 20 km/h
Tacómetro	Tacometro.jsx	0 - 8000 RPM	rpm	Indica las revoluciones del motor con marcas cada 1000 RPM (etiquetadas como 0-8)
Temperatura	Temperatura.jsx	40°C - 130°C	temperatura	Monitorea la temperatura del motor con marcas cada 20 °C
Voltaje	Voltaje.jsx	10V - 15V	voltaje	Muestra el voltaje del sistema eléctrico (batería/alternador) con marcas cada 1 V

Amperaje	Amperaje.jsx	-50A a +50A	amperaje	Indica carga (+) o descarga (-) de la batería con el 0 en el centro
Gasolina	Gasolina.jsx	0% - 100%	nivel	Muestra el nivel de combustible con etiquetas E, 1/4, 1/2, 3/4, F

Las características técnicas comunes de estos 6 componentes son:

- Usan SVG puro para dibujar arcos, agujas y marcas (sin librerías externas)
- Implementan la fórmula matemática: $\text{angulo} = 135 + (\text{porcentaje} * 270)$ para mapear valores a grados
- Incluyen animación CSS con transition y cubic-bezier para movimiento suave de la aguja
- Tienen degradados de color (verde → amarillo → rojo) que indican zonas seguras, de precaución y de peligro
- Muestran un display digital debajo del medidor con el valor exacto



2. Medidor digital:

Componente	Archivo	Props que Recibe	Función Específica
------------	---------	------------------	--------------------

Odómetro	Odometro.jsx	kilometrajeTotal, kilometrajeParcial	Muestra el kilometraje acumulado del vehículo (total) y el viaje actual (parcial)
-----------------	--------------	---	---

Las características técnicas de este componente son:

- No usa SVG ni agujas, es puramente texto formateado
- El Total se muestra en verde con 6 dígitos sin decimales
- El Parcial se muestra en amarillo con 1 decimal
- Usa Math.floor() y toFixed(1) para controlar los decimales
- Simula un display LCD con fondo negro y fuente monoespaciada

```

Odometro.jsx X
frontend > src > components > Odometro.jsx > Odometro
1  export default function Odometro({ kilometrajeTotal = 0, kilometrajeParcial = 0 }) {
2    // Formatear números: Total sin decimales, Parcial con 1 decimal
3    const formatKilometros = (km) => {
4      return Math.floor(km).toString().padStart(6, '0');
5    };
6
7    const formatParcial = (km) => {
8      return km.toFixed(1).padStart(5, '0');
9    };
10
11   return (
12     <div className="bg-gray-900 rounded-lg p-6 shadow-xl border border-gray-700">
13       <h2 className="text-white text-xl font-bold mb-4 tracking-widest">ODOMETRO</h2>
14
15       <div className="space-y-4">
16         {/* Kilometraje Total */}
17         <div className="bg-black rounded p-4 border border-gray-700">
18           <div className="text-gray-400 text-xs mb-1 text-center font-mono">TOTAL</div>
19           <div className="text-green-400 text-3xl font-mono font-bold">{formatKilometros(kilometrajeTotal)}</div>
20           <div className="text-gray-500 text-xs text-center mt-1">km</div>
21         </div>
22
23         {/* Kilometraje Parcial */}
24         <div className="bg-black rounded p-4 border border-gray-700">
25           <div className="text-gray-400 text-xs mb-1 text-center font-mono">PARCIAL</div>
26           <div className="text-yellow-400 text-3xl font-mono font-bold">{formatParcial(kilometrajeParcial)}</div>
27           <div className="text-gray-500 text-xs text-center mt-1">km</div>
28         </div>
29       </div>
30     </div>
31   );
32 }

```

3. Panel de Estado / Testigos (1 componente)

Componente	Archivo	Props que Recibe	Función Específica
LucesAdvertencia	LucesAdvertencia.jsx	7 props booleanas (checkEngine, aceite, frenoMano, direccionalIzq, direccionalDer, lucesAltas, lucesBajas)	Muestra 7 testigos luminosos que se encienden/apagan según el estado del vehículo

Las características técnicas de este componente son:

- Usa iconos SVG personalizados para cada tipo de luz (motor, aceite, freno, flechas, foco)
- Implementa un sistema de colores semánticos:
- Rojo (text-red-500): Peligro inmediato (Aceite, Freno de mano)
- Amarillo (text-yellow-500): Advertencia (Check Engine)
- Azul (text-blue-500): Informativo (Luces altas)
- Verde (text-green-500): Estado activo (Direccionales, Luces bajas)
- Aplica efecto glow/brillo con drop-shadow cuando la luz está activa
- Las luces apagadas se muestran en gris tenue (text-gray-700 opacity-30)

useVehicleData.js

En el ecosistema de React, este archivo se conoce como un Custom Hook (Hook Personalizado), su función es encapsular toda la lógica de negocio, la simulación de datos y la comunicación con el servidor, manteniéndola completamente separada de la parte visual (los componentes). Las funciones principales de este archivo son:

- Gestión del Estado (State Management): Mantiene en memoria todos los valores actuales del tablero (velocidad, RPM, temperatura, estado de las luces, etc.) usando useState. Cada vez que estos valores cambian, React actualiza automáticamente la interfaz gráfica.
- Motor de Simulación en Tiempo Real: Genera datos falsos pero coherentes cada 500 milisegundos. No usa números totalmente aleatorios; aplica fórmulas matemáticas para que el comportamiento sea realista (por ejemplo, las RPM suben si la velocidad sube, y la gasolina baja lentamente).
- Comunicación Asíncrona con el Backend: Cada 5 segundos, toma los datos actuales y los envía al servidor Node.js mediante una petición HTTP (fetch),

asegurando que la base de datos MongoDB se actualice constantemente sin saturar la red.

```

JS useVehicleData.js X
frontend > src > hooks > JS useVehicleData.js > useVehicleData
1 import { useState, useEffect, useRef } from 'react';
2
3 export function useVehicleData() {
4   const [data, setData] = useState({
5     velocidad: 0,
6     rpm: 800,
7     temperatura: 85,
8     voltaje: 13.8,
9     amperaje: 15.5,
10    gasolina: 75,
11    kilometraje: 45230,
12    kilometrajeParcial: 127.5,
13    checkEngine: false,
14    direccionalIzq: false,
15    direccionalDer: false,
16    lucesAltas: false,
17    lucesBajas: true,
18    aceite: false,
19    frenoMano: false
20  });
21
22  // Usamos un ref para tener siempre el valor más reciente de los
23  const dataRef = useRef(data);
24
25  // Actualizamos el ref cada vez que los datos cambian
26  useEffect(() => {
27    dataRef.current = data;
28  }, [data]);
29
30  useEffect(() => {
31    // 1. Simulación visual (se actualiza cada 500ms para que las a
32    const intervaloSimulacion = setInterval(() => {
33      setData(prev => {
34        const velocidadCambio = (Math.random() - 0.5) * 10;
35        let nuevaVelocidad = Math.max(0, Math.min(120, prev.velocidad
36          + velocidadCambio));
37        const rpmBase = 800 + (nuevaVelocidad / 120) * 5200;
38        let nuevaRpm = Math.max(800, Math.min(8000, rpmBase + (Math
39          .random() - 0.5) * 1000));
40        let nuevaTemp = Math.max(80, Math.min(100, prev.temperatura
41          + (Math.random() - 0.5) * 10));
42        let nuevoVolt = Math.max(13.5, Math.min(14.5, prev.voltaje
43          + (Math.random() - 0.5) * 0.5));
44        let nuevoAmp = Math.max(-10, Math.min(30, prev.amperaje + (
45          Math.random() - 0.5) * 10));
46        let nuevaGas = prev.gasolina - (Math.random() * 0.1);
47        if (nuevaGas < 0) nuevaGas = 100;
48
49        const kmAumento = (nuevaVelocidad / 3600) * 0.5;
50        const nuevoKmTotal = prev.kilometraje + kmAumento;
51        const nuevoKmParcial = prev.kilometrajeParcial + kmAumento;
52
53        return {
54          ...prev,
55          velocidad: nuevaVelocidad,
56          rpm: nuevaRpm,
57          temperatura: nuevaTemp,
58          voltaje: nuevoVolt,
59          amperaje: nuevoAmp,
60          gasolina: nuevaGas,
61          kilometraje: nuevoKmTotal,
62          kilometrajeParcial: nuevoKmParcial,
63          direccionalIzq: !prev.direccionalIzq // Parpadeo
64        };
65      });
66    }, 500);
67
68    // 2. Guardado en Base de Datos (se envía cada 5 segundos para
69    const intervaloGuardado = setInterval(() => {
70      fetch('http://localhost:3001/api/data/save', {
71        method: 'POST',
72        headers: { 'Content-Type': 'application/json' },
73        body: JSON.stringify(dataRef.current)
74      })
75        .then(response => response.json())
76        .then(resultado => console.log(' Guardado en BD:', resultado))
77        .catch(error => console.error(' Error al guardar:', error));
78    }, 5000); // Cada 5000 milisegundos (5 segundos)
79
80    // Limpiar intervalos al desmontar el componente
81    return () => {
82      clearInterval(intervaloSimulacion);
83      clearInterval(intervaloGuardado);
84    };
85  }, []);
86
87  return data;
88

```

App.jsx

Es el Componente Raíz (Root Component) de la aplicación, su función no es hacer cálculos ni dibujar medidores, sino unir todo: toma los datos que genera el hook y se los entrega a los componentes visuales, organizándolos en la pantalla mediante un diseño de cuadrícula. Las funciones principales de este archivo son:

- Centralización de Datos (Consumo del Hook): Es el único lugar en la aplicación que llama a `useVehicleData()`, suscribiéndose a los cambios y recibe el objeto `vehicleData` con toda la telemetría actualizada.
- Composición de la Interfaz (UI Layout): Utiliza clases de Tailwind CSS (`grid`, `grid-cols-1`, `md:grid-cols-2`, `gap-8`) para crear un diseño responsivo, para que los medidores se adaptan automáticamente si el usuario ve la página en un celular (1 columna), una tablet (2 columnas) o un monitor (3 columnas).

- Distribución de Props (Inyección de Dependencias): Toma los datos del objeto `vehicleData` y se los pasa a cada componente hijo mediante props. Por ejemplo, extrae `vehicleData.velocidad` y se la inyecta al componente `<Velocimetro />`.

```
App.jsx X
frontend > src > App.jsx > App
1  import Velocimetro from './components/Velocimetro';
2  import Tacometro from './components/Tacometro';
3  import Temperatura from './components/Temperatura';
4  import Voltaje from './components/Voltaje';
5  import Amperaje from './components/Amperaje';
6  import Gasolina from './components/Gasolina';
7  import LucesAdvertencia from './components/LucesAdvertencia';
8  import Odometro from './components/Odometro';
9  import { useVehicleData } from './hooks/useVehicleData';
10
11 function App() {
12   const vehicleData = useVehicleData();
13
14   return (
15     <div className="min-h-screen bg-gray-800 p-8">
16       <h1 className="text-white text-4xl font-bold text-center mb-8">
17         Dashboard Vehicular
18       </h1>
19
20       <div className="max-w-5xl mx-auto">
21         <LucesAdvertencia
22           checkEngine={vehicleData.checkEngine}
23           aceite={vehicleData.aceite}
24           frenoMano={vehicleData.frenoMano}
25           direccionalIzq={vehicleData.direccionalIzq}
26           direccionalDer={vehicleData.direccionalDer}
27           lucesAltas={vehicleData.lucesAltas}
28           lucesBajas={vehicleData.lucesBajas}
29         />
30       </div>
31
32       <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
33         <Velocimetro velocidad={vehicleData.velocidad} />
34         <Tacometro rpm={vehicleData.rpm} />
35         <Temperatura temperatura={vehicleData.temperatura} />
36         <Voltaje voltaje={vehicleData.voltaje} />
37         <Amperaje amperaje={vehicleData.amperaje} />
38         <Gasolina nivel={vehicleData.gasolina} />
39         <Odometro
40           kilometrajeTotal={vehicleData.kilometraje}
41           kilometrajeParcial={vehicleData.kilometrajeParcial}
42         />
43       </div>
44     </div>
45   );
46 }
47
```